**TECNOLÓGICO NACIONAL DE MEXICO**
**INSTITUTO TECNOLÓGICO DE TIJUANA**

**SUBDIRECCIÓN ACADÉMICA**
**DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

**SEMESTRE:**
ENERO - JUNIO 2026

**NOMBRE DE LA CARRERA:**
INGENIERÍA EN SISTEMAS COMPUTACIONALES

**NOMBRE DE LA MATERIA:**
PATRONES DE DISEÑO DE SOFTWARE

**TÍTULO DEL TRABAJO:**
SINGLETON

**UNIDAD A EVALUAR:**
UNIDAD 2

**NOMBRE DEL ALUMNO:**
RODRIGUEZ MORALES SUSANA ABIGAIL - 22210346

**NOMBRE DEL MAESTRO:**
MARIBEL GUERRERO LUIS

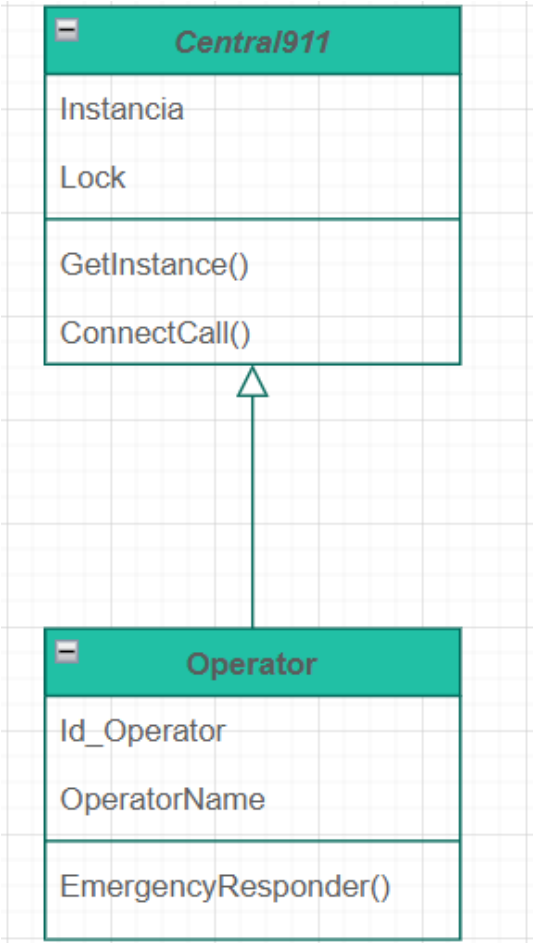
# Índice


Singleton 3
Diagrama UML 3
Código 4
Central 911 4
Operador 5
Cliente 6
Resultado 7
Identificar y explicar 8
1.- El atributo estático de la instancia 8
Clase Central 911 8
2.- El constructor 8
Clase Central 911 8
Clase Operador 8
¿Por qué no se pueden crear múltiples objetos? 9
Conclusión 9

## Singleton

Este patrón garantiza que una clase tenga una única instancia, teniendo de un punto de acceso de manera global, de modo que restringe la creación de nuevas instancias de la clase.

## Diagrama UML



# Código

## Central 911

```
namespace Singleton
{
    //accesibilidad desde cualquier parte
    6 referencias
    public class Central911
    {
        //guarda la única instancia de la clase
        //crea un objeto que despues de asignarse no puede modificarse (readonly)
        private static Central911 _instance;

        //evita que se creen más instancias
        private static readonly object _lock = new object();

        //variable que permite leer un valor y el otro solo puede ser modificado por la clase
        1 referencia
        public string Central { get ; private set;}

        //constructor privado
        1 referencia
        private Central911()
        {
            Central = "Central 911"; //nombre de la instancia (a donde se esta comunicando)
        }

        //metodo que devuelve la instancia
```

```
        //metodo que devuelve la instancia
        1 referencia
        public static Central911 GetInstance()
        {
            //verifica si no existe la instancia
            if (_instance == null)
            {
                //bloquea la creación de más instancias
                lock (_lock)
                {
                    //doble verificacion de que no exista la instancia
                    if (_instance == null)
                    {
                        //si no existe la crea
                        _instance = new Central911();
                    }
                }
            }
            //regresa la instancia, ya sea recien creada o la que ya estaba
            return _instance;
        }

        //método que conecta a una llamada con parametros de operador y tipo de emergencia
```

```
        //método que conecta a una llamada con parametros de operador y tipo de emergencia
        1 referencia
        public void ConnectCall(Operator operador, string EmergencyType)
        {
            //se muestra el nombre del operador que está atendiendo
            Console.WriteLine("\nLlamada conectada con el operador: " + operador.Operator_Name);
            //Muestra el tipo de emergencia
            operador.EmergencyResponder(EmergencyType);
        }
    }
}
```

## Operador

```
namespace Singleton
{
    //define publica la clase
    12 referencias
    public class Operator
    {
        //variable para guardar los datos del operador
        1 referencia
        public int Id_Operator { get; set; } //guarda el identificador
        3 referencias
        public string Operator_Name { get; set; } //guarda el nombre del operador


        //constructor
        8 referencias
        public Operator(int id, string name)
        {
            Id_Operator = id; //identificador del operador
            Operator_Name = name; //nombre del operador
        }

        //método que contiene los tipos de emergencia o situaciones
```

```
        //método que contiene los tipos de emergencia o situaciones
        1 referencia
        public void EmergencyResponder(string EmergencyType)
        {
            //menciona quién atiende y de qué tipo de emergencia se trata
            Console.WriteLine($"Operador {Operator_Name} atendiendo emergencia del tipo: {EmergencyType}");

            //distintos casos para los tipos de emergencia
            switch (EmergencyType)
            {
                case "Intento de suicidio":
                    Console.WriteLine("Enviando unidades de apoyo y rescate.");
                    break;

                case "Incendio":
                    Console.WriteLine("Enviando bomberos.");
                    break;

                case "Accidente":
                    Console.WriteLine("Enviando paramédicos y oficiales.");
                    break;

                case "Violeta":
                    Console.WriteLine("Enviando unidad de policia violeta.");
                    break;
                case "Robo":
                    Console.WriteLine("Enviando unidad de policia para investigación");
                    break;
                default:
                    Console.WriteLine("Tipo de emergencia no reconocido.");
                    break;
            }
        }
    }
}
```

## Cliente

```
namespace Singleton
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main(string[] args)
        {
            //random para elegir indice de lista y asignar un operador a la llamada
            Random rnd = new Random();

            //se crean la llamadas que hace uso de la instancia
            Central911 Llamada = Central911.GetInstance();

            //se crea una lista y se guardan los operadores
            List<Operator> operators = new List<Operator>
            {
                new Operator(1, "Gerardo"),
                new Operator(2, "Carlos"),
                new Operator(3, "Laura"),
                new Operator(4, "Felicia"),
                new Operator(5, "Bruno"),
                new Operator(6, "Esteban"),
                new Operator(7, "Rocio"),
                new Operator(8, "María")
            };

            //Lista para las emergencias
```

```
            //Lista para las emergencias
            List<String> emergency = new List<string>
            {
                "Violeta",
                "Intento de suicidio",
                "Incendio",
                "Accidente",
                "Robo"
            };

            //simulación de varias llamadas
            //el indice del operador se selecciona de manera random desde la cantidad de operadores existentes
            //y se asigna de manera aleatoria una situación
            for (int i = 0; i < 5; i++)
            {
                Llamada.ConnectCall(operators[rnd.Next(operators.Count)], emergency[rnd.Next(emergency.Count)]);
                Console.WriteLine(Llamada.GetHashCode()); //muestra el identificaor del objeto
            }

            Console.ReadKey();
        }
    }
}
```

Se agregaron dos listas, una para los operadores y otra para las emergencias.
Para las llamadas de manera random se seleccionaron los operadores y las situaciones de emergencias que se iban a mostrar. Para no hacer manual las llamadas se colocaron en un ciclo que generaba una nueva llamada lo que garantiza el uso de la misma instancia, aún así se incluyó el método Hashcode para comprobar que era la misma instancia.

## Resultado

```
C:\Users\sussa\OneDrive\Doc

Llamada conectada con el operador: Bruno
Operador Bruno atendiendo emergencia del tipo: Incendio
Enviando bomberos.
46104728

Llamada conectada con el operador: María
Operador María atendiendo emergencia del tipo: Intento de suicidio
Enviando unidades de apoyo y rescate.
46104728

Llamada conectada con el operador: Esteban
Operador Esteban atendiendo emergencia del tipo: Robo
Enviando unidad de policia para investigación
46104728

Llamada conectada con el operador: Felicia
Operador Felicia atendiendo emergencia del tipo: Accidente
Enviando paramédicos y oficiales.
46104728

Llamada conectada con el operador: Bruno
Operador Bruno atendiendo emergencia del tipo: Violeta
Enviando unidad de policia violeta.
46104728
```

# Identificar y explicar

## 1.- El atributo estático de la instancia

### Clase Central 911

Se declara una variable que contendrá la instancia, y los modificadores de tipo privado y estático están indicando que el elemento solo será accesible desde la misma clase y que otros no puedan acceder a ella.

```
private static Central911 _instance;
```

En la declaración de este atributo, los modificadores indican que solo existe para toda la clase, solo va a ser usado por esa clase, no va a poder ser reasignado una vez generado y garantizando que solo sea usado un elemento.

```
private static readonly object _lock = new object();
```

## 2.- El constructor

Un constructor es un método que de manera automática se ejecuta al ser creada una instancia de una clase.

### Clase Central 911

Al ser privado, permite restringir la creación de instancias fuera de la clase, y asignando un nombre, ideal para el patrón singleton ya que no se va a crear otra instancia.

```
1 referencia
private Central911()
{
    Central = "Central 911";
}
```

### Clase Operador

Aquí es público, recibe los parámetros del operador y se van a poder compartir entre clases, para poder asignarles en este caso una llamada.

```
2 referencias
public Operator(int id, string name)
{
    Id_Operator = id; //
    Operator_Name = name; //
}
```

## ¿Por qué no se pueden crear múltiples objetos?

Porque se está haciendo uso del patrón singleton y lo que hace es que garantiza el una única instancia, y para ello se creó un atributo que bloqueaba la creación de otras instancias.

## Conclusión

Gracias al patrón se puede tener una mayor gestión de las veces que se hace uso de un mismo recurso, evitando desastres entre los interesados en comunicarse.